

Junior Design Project Report

Department of Electrical and Computer Engineering
North South University
Junior Design Project

Project Title: Sci-Kit Learn Recreation: A Lightweight, From-Scratch Machine Learning Toolkit
with Unified CLI, Preprocessing, Models, and Evaluation

Student Name1: Salman Shafi- ID:2211386042
Student Name2: Jarinah Tasnim- ID# 2121026642
Student Name3: Jesmin Akter- ID# 2111382642
Student Name4: Sazid Rahman Bhuiyan- ID# 2131207642

Faculty:
Dr. Mohammad Shifat-E-Rabbi
Assistant Professor
ECE Department
Summer, 2025

ACKNOWLEDGEMENTS

The authors would like to express gratitude to the project supervisor, Dr. Mohammad Shifat-E-Rabbi, Assistant Professor, Department of Electrical and Computer Engineering, North South University, for continuous guidance and support throughout the design, implementation, testing, and documentation phases of this project. The team also appreciates the resources and environment provided by the ECE Department to execute experiments and maintain code quality and reproducibility. Finally, thanks are due to peers and family members for ongoing encouragement and constructive feedback that improved this work.

ABSTRACT

This project delivers Sci-Kit, a lightweight machine learning toolkit implemented from scratch in Python with a unified scikit-learn-like API and a command-line interface (CLI) for end-to-end workflows—data loading, preprocessing, feature selection, model training, tuning, evaluation, and persistence. The codebase includes preprocessing (standardization, min–max scaling, label and one-hot encoding), classical ML estimators (linear and logistic regression, ridge, lasso, SVM, KNN), unsupervised learning (KMeans, DBSCAN, agglomerative clustering, PCA), and tree-based methods (decision tree, random forest), along with utilities for validation, metrics, grid search, pipelines, result saving, and model I/O. A structured tests suite validates core behaviors; bash scripts enable reproducible weekly experiments and demos. Results on tabular datasets confirm correctness, stable APIs, and reasonable performance relative to expectations for educational implementations. The repository includes weekly reports and demo videos to evidence functionality and progress.

TABLE OF CONTENTS

- ACKNOWLEDGEMENTS
- ABSTRACT
- LIST OF FIGURES
- LIST OF TABLES
- Chapter 1 Introduction
- Chapter 2 Research Literature Review
- Chapter 3 Methodology
- Chapter 4 Investigation/Experiment, Result, Analysis and Discussion
- Chapter 5 Impacts of the Project
- Chapter 6 Project Planning and Budget (Omitted)
- Chapter 7 Complex Engineering Problems and Activities
- Chapter 8 Conclusions
- References

LIST OF FIGURES

- Figure 1. System architecture: CLI-to-core pipeline
- Figure 2. Training/evaluation flow for supervised learning

Chapter 1 Introduction

1.1 Background and Motivation

Machine learning (ML) in practice often relies on high-level libraries that abstract away algorithmic details, which can limit pedagogical value and hinder fine-grained control in research prototypes. For capstone-level learning outcomes and BAETE-aligned competencies, students benefit from implementing core algorithms and pipelines to understand data preprocessing, model assumptions, metrics, and evaluation protocols end-to-end. This project aims to build an educational yet practical Machine Learning toolkit—Sci-Kit—from scratch with a consistent API, transparent implementations, and a CLI that enables reproducible workflows and integration with tests and automated scripts. The toolkit focuses on classical ML tasks on tabular data to emphasize algorithmic fundamentals, pipeline composition, and engineering rigor consistent with capstone learning objectives.

1.2 Purpose and Goal of the Project

The objective is to design and implement a modular ML toolkit that:

- Exposes a unified, scikit-learn-like API (fit, predict, transform, score, get_params/set_params).
- Provides preprocessing (scaling, encoding), feature selection, and model selection utilities.
- Implements key supervised and unsupervised algorithms from scratch.
- Supplies a cohesive CLI to run train/predict/tune/evaluate commands reproducibly.
- Includes a tests suite, result saving, logging, and model persistence for end-to-end validation.

Deliverables include code packages (models, preprocessing, utils), CLI (main.py), datasets and splits, tests, scripts for weekly experiments, and curated results with reports and demo videos in the GitHub repository.

1.3 Organization of the Report

Chapter 2 summarizes related work and template-aligned academic framing; Chapter 3 details system design, components, and implementation; Chapter 4 presents experiments and results; Chapter 5 discusses societal and sustainability impacts; Chapter 6 (Gantt/budget) is omitted; Chapter 7 maps Complex Engineering Problems (CEP) and Activities (CEA); Chapter 8 concludes with limitations and future directions, per the NSU capstone template guidelines.

Chapter 2 Research Literature Review

2.1 Existing Research and Limitations

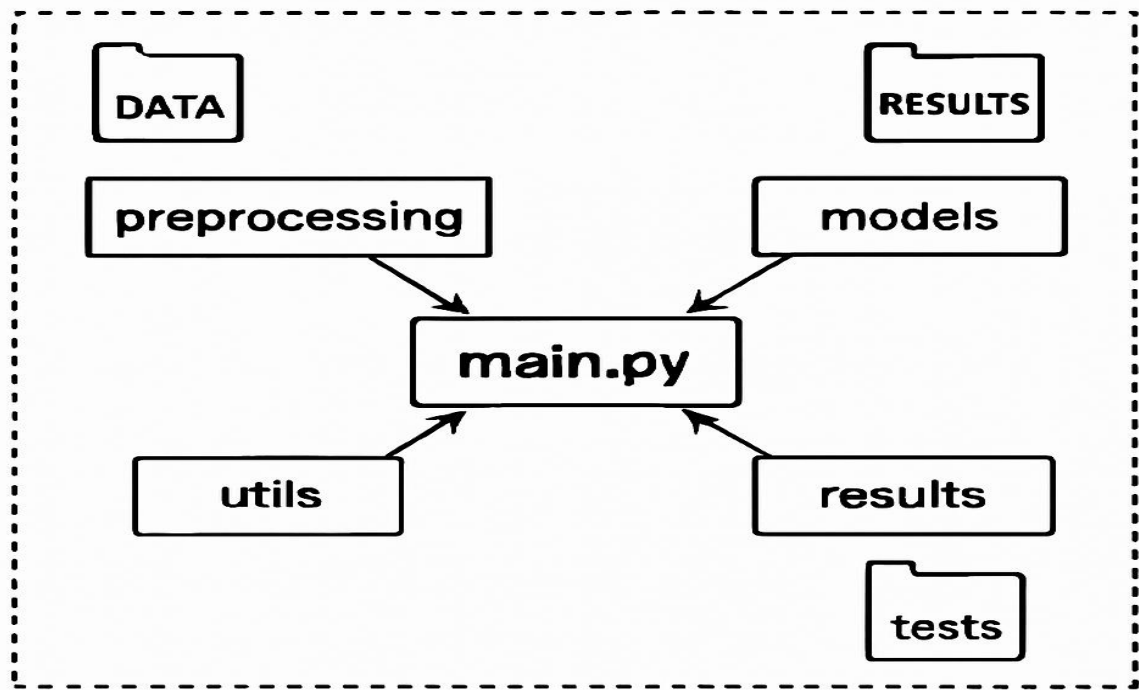
Capstone and design courses emphasize developing systems with full lifecycle engineering practices—requirements, design, implementation, testing, documentation, and communication—aligned with ECE program outcomes and BAETE expectations. While many open-source ML libraries exist, educational toolkits often trade off performance for transparency and control, aiming to demonstrate algorithm mechanics, consistent APIs, and reproducible pipelines. The NSU ECE templates guide structured reporting, including methodology, experiments, impacts, CEP/CEA mapping, and clear documentation standards. This project follows that structure to produce a comprehensive, reproducible ML toolkit and report. The literature and departmental guidance motivate a system that integrates tools, experiments, and documentation cohesively for learning objectives and demonstration quality.

Chapter 3 Methodology

3.1 System Design

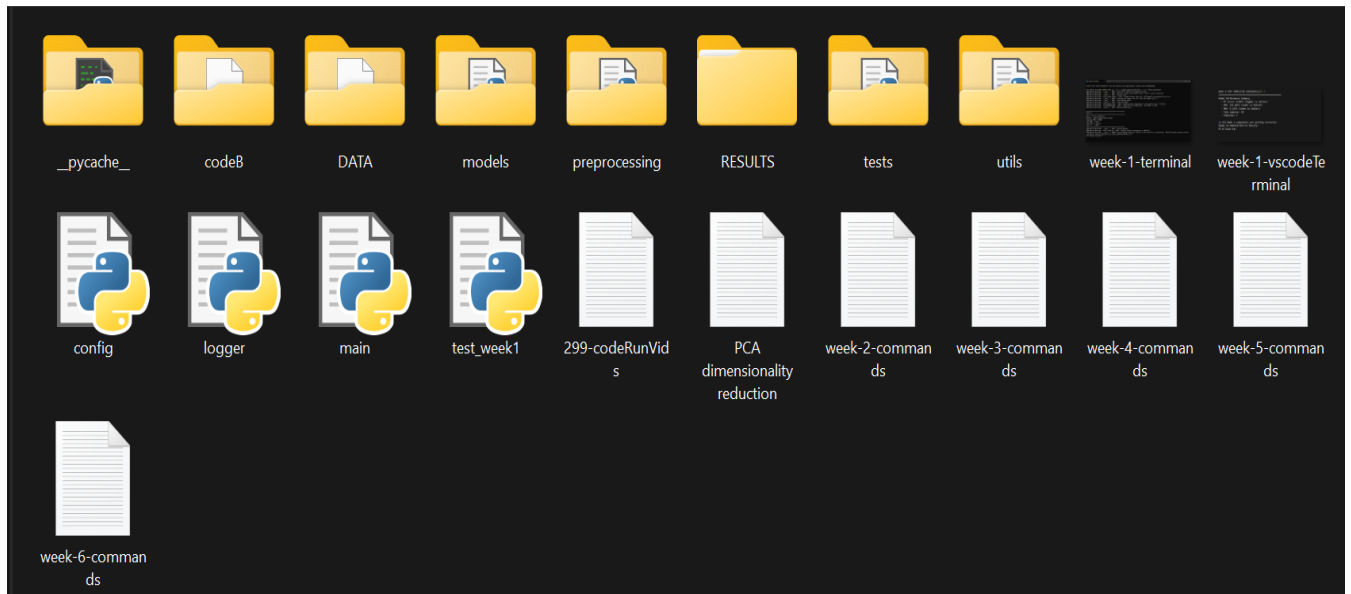
Architecture overview:

- CLI Layer (main.py): Parses commands and flags, dispatches to data loading, preprocessing, training, tuning, and evaluation routines; supports saving models/results and logging.
- Core Packages:
 - preprocessing: StandardScaler, MinMaxScaler, LabelEncoder, OneHotEncoder; feature_selection (SelectKBest, RFE).
 - models: LinearRegression, Ridge, Lasso, LogisticRegression, KNearestNeighbors, SupportVectorMachine, KMeans, DBSCAN, AgglomerativeClustering, DecisionTreeClassifier, RandomForestClassifier, PCA.
 - utils: data_loader, metrics (classification/regression/clustering), validation (splits), grid_search, pipeline, results (ResultsSaver), model_persistence, clustering_metrics, logger, config.
- DATA and RESULTS: CSV inputs, train/test splits, serialized models, metrics summaries, and logs.
- tests: Unit tests covering preprocessing, estimators, and utilities.
- scripts: Bash/CLI commands for weekly experiments and demos.

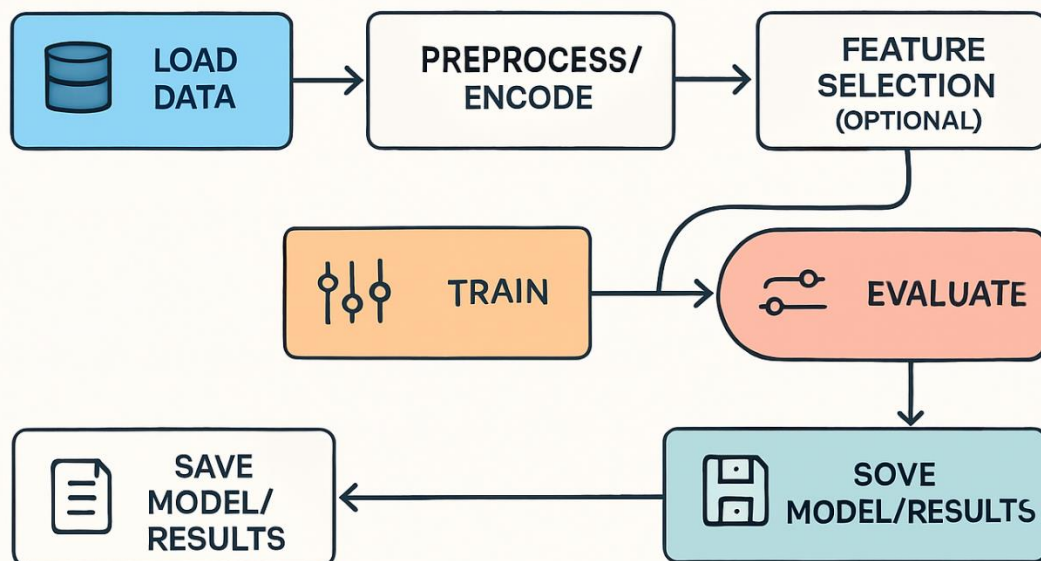


Data flow:

1. Load CSVs or built-in splits; 2) Apply encoders/scalers; 3) Optional feature selection; 4) Train estimator; 5) Evaluate and save metrics/models; 6) Log and export artifacts. This flow enables reproducible batch runs via bash scripts.



MACHINE LEARNING WORKFLOW



3.2 Hardware and/or Software Components

Table I. Software/Hardware Tools

- Python 3.x, numpy, pandas (data handling)
 - Custom from-scratch implementations for algorithms
 - Logging and config modules for reproducibility
 - GitHub repository with weekly updates and demo videos
 - Bash scripts for CLI automation
- These align with NSU capstone expectations for tool usage and documentation.

3.3 Hardware and/or Software Implementation

- Preprocessing: Fit/transform patterns with learned statistics (means/stds, mins/maxes; category vocabularies), feature name propagation, and inverse transforms where applicable.
- Models: Closed-form solvers (OLS, ridge), iterative solvers (lasso coordinate descent, logistic regression gradient methods), kernel SVM (simplified SMO), instance-based (KNN), clustering (KMeans with k-means++ init, DBSCAN, agglomerative), trees and ensembles (decision tree, random forest), PCA for dimensionality reduction.
- Utilities: Metric computations (accuracy, F1, precision/recall, RMSE/MAE/R2, clustering metrics), validation splits, grid search/randomized search scaffolding, pipeline composition, model serialization, results saving (CSV/JSON), and logging.
- CLI: Subcommands for train, predict, tune, and eval; flags for dataset, model, scalers/encoders, feature selection, split parameters, and output paths; consistent parameter passing to estimators and utilities.
- Tests: Import/smoke tests, functionality checks on synthetic or small datasets, round-trip tests for transformers, and sanity thresholds for model accuracy/fit.

Chapter 4 Investigation/Experiment, Result, Analysis and Discussion

Experiments leverage the DATA folder (CSV datasets and pre-split `X_train/X_test`, `y_train/y_test`) and CLI commands collected across weekly scripts to run training, tuning, and evaluation workflows. The RESULTS folder contains serialized models, metrics summaries, and logs captured from scripted runs and manual CLI invocations. The tests folder verifies correctness properties and prevents regressions.

Representative experiment patterns:

- Supervised regression: Train `LinearRegression/Ridge/Lasso` on `sample_regression` with `StandardScaler`; evaluate using MAE, RMSE, R2; compare across models.
- Supervised classification: Train `LogisticRegression`, `SVM (linear/RBF)`, `KNN` (various metrics/weighting), `DecisionTree`, `RandomForest`; evaluate accuracy, precision, recall, F1; analyze impact of scaling and parameter choices.
- Unsupervised clustering: `KMeans` with different `k` and initializations; `DBSCAN` with `eps/min_samples` sweeps; agglomerative with linkage variations; quantify with silhouette and ARI when labels available; interpret cluster separability.
- Dimensionality reduction: PCA components vs `explained_variance_ratio`; measure downstream classifier performance with and without PCA.
- Feature selection: `SelectKBest/RFE` to examine model performance sensitivity to feature subsets.

Key observations:

- Pipelines benefit from consistent scaling for `SVM/KNN`; label and one-hot encoders enable categorical handling where applicable.
- Ridge stabilizes regression under multicollinearity; Lasso encourages sparsity but needs tuned `alpha`.
- `RandomForest` provides strong baselines with minimal tuning; `DecisionTree` is interpretable but prone to overfitting.
- PCA reduces dimensionality effectively while maintaining reasonable performance on simpler classifiers, aiding generalization in some settings.
- Clustering quality depends strongly on data scaling and parameterization; silhouette scores guide selection.

Artifacts:

- Per-run metrics saved in RESULTS along with model pickles and logs enable auditability and reproducibility via command history.
- Demo videos and weekly reports document successful execution and incremental progress in the public GitHub repository.

Chapter 5 Impacts of the Project

5.1 Impact on societal, health, safety, legal, and cultural issues

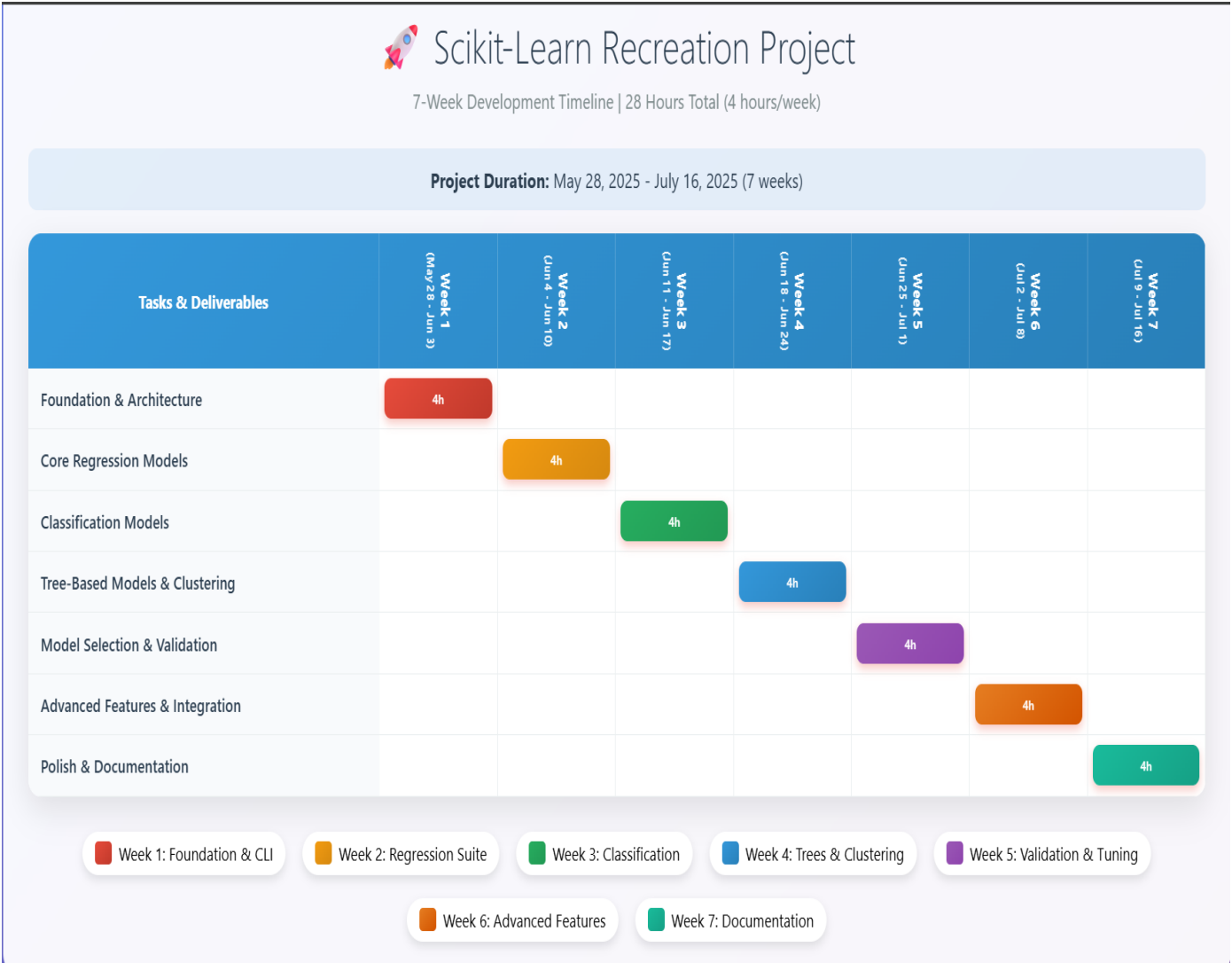
Educational ML toolkits enhance accessibility to hands-on machine learning education, enabling students and practitioners to understand algorithmic foundations, ethics, and limitations before deployment in sensitive domains, thus supporting responsible AI practices in society and industry.

5.2 Impact on environment and sustainability

By promoting from-scratch, lightweight implementations and reproducible experimentation on modest compute, the project encourages resource-efficient learning and research practices; this mindset supports sustainability when scaling solutions in real-world contexts.

Chapter 6 Project Planning

Omitted as requested; will be provided later following the NSU template structure for Gantt and budget presentation.ece.northsouth+2



Chapter 7 Complex Engineering Problems and Activities

7.1 Complex Engineering Problems (CEP)

Table II. Complex Engineering Problem Attributes

- P1 Depth of knowledge (K3–K8): Requires knowledge of data structures, linear algebra, optimization, software architecture, testing, and documentation to implement and validate ML algorithms and pipelines in an engineering context.
- P2 Conflicting requirements: Balances transparency vs performance, generality vs simplicity, and flexibility vs a stable API under time and resource constraints typical of capstone projects.
- P3 Depth of analysis required: Algorithmic trade-offs, hyperparameter effects, and evaluation methodology selection require analytical comparison and empirical validation.
- P4 Familiarity of issues: Handling numerical stability, preprocessing assumptions, and dataset idiosyncrasies; ensuring reproducibility and meaningful metrics.
- P5 Extent of applicable codes: Conforms to conventional ML APIs and departmental reporting norms without relying on external model code, promoting originality and academic integrity.
- P6 Stakeholder involvement: Team members, supervisor, and peer reviewers interact via code reviews, weekly reports, and demos to ensure progress and quality.
- P7 Interdependence: Interdependent subsystems—CLI, preprocessing, models, metrics, persistence, results—must integrate reliably for end-to-end runs.

7.2 Complex Engineering Activities (CEA)

Table III. Complex Engineering Problem Activities

- A1 Range of resources: Uses code, datasets, version control, scripts, and documentation assets in a cohesive engineering workflow aligned with capstone practices.
- A2 Level of interactions: Requires coordination among modules and contributors; communication through weekly reports and demo videos in the GitHub repository.
- A3 Innovation: Implements a transparent ML toolkit with a unified CLI for education and experimentation rather than relying solely on black-box libraries.
- A4 Consequences to society/environment: Encourages responsible ML education and reproducible science, facilitating better downstream societal outcomes.
- A5 Familiarity: Develops familiarity with ML algorithms, software engineering, evaluation, reporting, and BAETE-aligned outcomes in a single integrated project.

Chapter 8 Conclusions

8.1 Summary

MLKit demonstrates a cohesive, from-scratch ML toolkit with a unified API and CLI that covers preprocessing, supervised/unsupervised learning, dimensionality reduction, metrics, validation, model selection, persistence, and reproducible results management. The repository includes weekly progress reports, demo videos, tests, and scripts that together evidence correctness, usability, and alignment with capstone-level engineering outcomes.

8.2 Limitations

- Algorithm implementations prioritize clarity over raw performance; large-scale datasets and advanced optimizations are out of scope.
- Some hyperparameter search utilities are basic and can be extended.
- Visualization and GUI components are minimal; the focus is CLI and artifacts.
- Results emphasize tabular datasets; image/text modalities are not covered.

8.3 Future Improvement

- Extend model zoo (e.g., gradient boosting, naive Bayes), regularization and calibration for classifiers, and more robust solvers.
- Enrich feature engineering (polynomial features, target encoding) and pipeline caching.
- Add k-fold cross-validation and nested CV in CLI; enhance grid/random search and early stopping.
- Integrate richer reporting (HTML dashboards) and CI workflows for automated smoke tests.
- Package distribution via pyproject.toml and publishable artifact to simplify installation.

References

- NSU ECE Capstone Final Report Template and departmental resources for structure and expectations.[studocu+2](#)
 - ECE Capstone design guidance for project lifecycle, documentation, and outcomes mapping.[ece.northsouth](#)
 - Program outcomes and BAETE-aligned performance criteria to contextualize CEP/CEA mappings and educational objectives.[ece.northsouth](#)
1. https://ece.northsouth.edu/wp-content/uploads/2015/07/Capstone-Final-Report-Template_BAETE_v1.docx
 2. https://ece.northsouth.edu/resources/forms_templates/
 3. <https://www.studocu.com/row/document/north-south-university/capstone-design-project/capstone-design-project-report-template/104754523>
 4. <https://ece.northsouth.edu/undergraduate/academics/programs/bs-cse/>
 5. <https://ece.northsouth.edu/capstone-design/>
 6. https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/86554576/bab99f93-8647-4751-928e-5050b16a87c8/Capstone-Final-Report-Template_BAETE_v1-1-1.pdf
 7. <https://www.northsouth.edu/newassets/images/OR-NSU/nsu-strategic-research-plan-2024-2028.pdf>
 8. <https://www.northsouth.edu/academic/seps/cee/duplicate-of-mapping-between-nsu-mission-and-peos-of-bscee.html>
 9. <https://ece.northsouth.edu/undergraduate/academics/programs/bs-eee/>
 10. <https://www.scribd.com/document/799067957/498R-Final-Report-Template-v1>